

AI Growth Automation System

Vue d'ensemble detaillee des 7 agents

Roles, entrees/sorties, et analyse verifiee contre le code reel

Prepare a partir du roadmap officiel (agent_dev_roadmap_v3), du work plan de l'Analyst, et d'une revue directe du code source de chaque agent (pas seulement de leur documentation).

2026-08-02

Recapitulatif : conformite au format de log obligatoire

Chaque agent doit ecrire ses actions dans un format JSON obligatoire a 9 champs (agent_name, action_type, input_summary, output_summary, lead_id, client_id, model_used, latency_ms, timestamp). Verifie directement dans le code de chacun, pas seulement dans leur documentation.

Analyst: CONFORME Format exact, JSONL + table Postgres agent_logs, teste.

CRM Keeper: CONFORME Format exact via _log_action(), JSONL append-only bien fait.

Media Buyer: NON CONFORME Construit le dict correct mais ne l'ecrit jamais (juste un print()).

Closer: NON CONFORME Ne construit jamais le format obligatoire, nulle part dans le code.

Commander: NON CONFORME Champs faux/manquants (pas de lead_id/input_summary/output_summary constants).

Content Strategist: NON CONFORME Schema totalement different (client_id/version/asset_type), README vide.

Qualifier: NON CONFORME Il manque input_summary, output_summary, client_id et latency_ms.

Seuls 2 agents sur 7 respectent le contrat de logging du cahier des charges.

Agent 1 : Commander

Couche : Acquisition (au-dessus de toutes les couches)

Definition du role

Le Commander est le cerveau du systeme. Il ne fait aucun travail marketing lui-meme : il recoit tous les evenements entrants (Telegram, webhooks), classe l'intention via un LLM, decide quel agent doit traiter la tache, et resout les conflits entre agents selon une hierarchie de priorite stricte (operateur humain > donnees Analyst > score Qualifier > etat CRM Keeper > jugement propre du LLM).

Responsabilites

- Recevoir tous les evenements systeme et les router vers le bon agent.
- Maintenir un registre en direct de tous les agents (statut, derniere action).
- Resoudre les conflits inter-agents selon la hierarchie de priorite.
- Generer un digest quotidien pour l'operateur.
- Stocker/recuperer la configuration globale par client (multi-tenant).

Ce que cet agent ne fait PAS

- N'ecrit jamais de copie publicitaire, ne score jamais un lead, n'envoie jamais de message.
- Ne prend pas de decision d'achat media : il instruit le Media Buyer, ne l'execute pas.
- N'evalue pas la qualite d'un contenu, il le route seulement.

Exemple d'entree (input)

```
POST /event
{
  "event_type": "new_lead",
  "target_agent": "qualifier",
  "raw_text": "Nouveau lead recu via Typeform",
  "priority": 2,
  "client_id": "client_a"
}
```

Exemple de sortie (output)

```
{
  "target": "qualifier",
  "status": "success"
}
```

(ou en cas d'echec repete : "status": "failed" apres 1 retry automatique)

Analyse detaillee (verifiee contre le code reel)

Phase C affichee dans le README (multi-client, memoire operateur, LangChain + DeepAgents + FastAgents + CrewAI, detection d'anomalies, Langfuse). Le code reel est bien plus avance fonctionnellement que les autres agents.

Logging : non conforme au format obligatoire. `logger.py::log_event()` ecrit {agent_name, client_id, action_type, event_type, priority, target_agent, reasoning, raw_text} -- il manque input_summary, output_summary et lead_id. Le stockage n'est pas non plus append-only : events.json est relu et reecrit en entier a chaque log.

Code mort : environ 430 lignes d'une premiere version complete de CommanderAgent sont encapsulees dans un bloc triple-quote jamais execute, laissees dans core/agent.py actif.

Resilience : core/health.py ne fait qu'un seul retry manuel avec un except: nu (attrape tout, y compris les interruptions) plutot qu'un retry avec backoff (tenacity). Pas de httpx non plus, requests synchrone est utilise dans un contexte FastAPI async.

Routes reelles verifiees : POST /event (route generique vers un agent cible, ne recoit pas de rapport adresse a lui-meme), POST /text (passe par le classificateur LLM + memoire operateur), GET /status (verifie /health de chaque agent enregistre).

Consequence pour l'Analyst : le rapport hebdomadaire ne peut pas etre livre proprement au Commander avec l'API actuelle -- ni /event ni /text ne sont concus pour recevoir un rapport structure. Reste sur Telegram en attendant un endpoint d'intake dedie cote Commander.

Agent 2 : CRM Keeper

Couche : Intelligence

Definition du role

Le CRM Keeper est la memoire du systeme entier. Contrairement aux autres agents, qui sont stateless, il connait chaque lead, son historique, son score, sa position dans le pipeline. Aucun agent n'a le droit de toucher le CRM directement : tout passe par lui, ce qui garantit la validation des donnees et empeche les doublons.

Responsabilites

- Creer un nouveau lead a la soumission d'un formulaire (UUID unique).
- Mettre a jour un lead quand un agent effectue une action.
- Fournir les donnees de lead aux autres agents en JSON structure uniquement.
- Valider chaque champ via Pydantic avant ecriture, rejeter les donnees invalides.
- Gerer plusieurs CRM (HubSpot, User.com) via une architecture d'adaptateurs (CRMFactory).
- Bloquer les retours en arriere de pipeline illogiques (override manuel possible).
- Gerer le tagging de segments et la recuperation en masse par etape/segment.

Ce que cet agent ne fait PAS

- Ne route pas les taches (role du Commander).
- Ne score pas les leads (role du Qualifier).
- Ne communique jamais directement avec un prospect.
- Ne cree pas de campagnes publicitaires.

Exemple d'entree (input)

```
GET /crm/leads/stage/closed_won?client_id=client_1
```

Exemple de sortie (output)

```
[
  {
    "lead_id": "hs-12345",
    "client_id": "client_1",
    "lead_stage": "closed_won",
    "qualification_score": 82,
    "source": "facebook_ads",
    "tags": ["high_value_real_estate"],
    "data_quality_score": 100,
    "gdpr_status": "active"
  }
]
```

(reponse LeadResponse -- aucun champ de valeur de deal/contrat)

Analyse detaillee (verifiee contre le code reel)

Le SEUL agent, avec l'Analyst, dont le logging est reellement conforme au format obligatoire -- verifie a la fois dans le code (agent.py::_log_action) et dans les vrais logs deja produits (crm_keeper_logs.json), en JSONL append-only correctement fait.

Routes reelles verifiees (port 8000) : GET /crm/leads/stage/{stage_name}, GET /crm/leads/{lead_id}, GET /crm/leads/cold, POST /crm/update, POST /crm/leads, PUT /crm/leads/{lead_id}, POST /marketing/audiences/sync. Aucune authentication sur aucune route.

Limitation structurelle majeure : le modele LeadResponse ne contient AUCUN champ de valeur monetaire (pas de contract_value, deal_value ou equivalent). Consequence directe : le calcul du ROAS par l'Analyst est structurellement impossible a partir de cette API tant que ce champ n'est pas ajoute.

GET /crm/leads/cold exclut deliberement les etapes closed_won/closed_lost/meeting_booked -- incompatible par construction avec toute analyse de deals fermes.

Bug mineur trouve : GET /crm/leads/{lead_id} reinitialise toujours created_at a 'maintenant' au lieu de retourner la vraie date de creation.

C'est l'agent le plus solide techniquement des 6 revus, avec le README le plus complet (Phase C : multi-tenant, audit Langfuse, scoring qualite NLP, blocage RGPD HTTP 451).

Agent 3 : Qualifier

Couche : Conversion

Definition du role

Le Qualifier est le filtre du systeme. Il fait passer chaque lead entrant par un score 0-100 sur 5 dimensions (budget, timing, autorite de decision, adequation probleme-solution, qualite d'engagement), combinant regles deterministes et analyse LLM, puis classe en disqualifie / MQL / SQL selon des seuils fixes.

Responsabilites

- Evaluer la qualite des leads via formulaires (Typeform) ou conversations.
- [Phase C] Comparer ses predictions SQL aux ventes reellement conclues (feedback loop).
- [Phase C] Faire tourner des tests A/B de flux de questions.
- [Phase C] Re-scoring en temps reel un lead qui revisite une page a forte intention.
- [Phase C] Enrichir les leads B2B via Apollo.io/Clearbit avant scoring.
- [Phase C] Evaluation hebdomadaire de precision poussee dans Langfuse.

Ce que cet agent ne fait PAS

- Ne stocke aucun historique permanent -- tout passe par le CRM Keeper.
- N'interagit jamais directement avec un prospect (pas d'email, pas de WhatsApp).
- Ne synchronise pas de segments vers les plateformes publicitaires.

Exemple d'entree (input)

```
POST /form-submission
{
  "client_id": "client_1",
  "industry_key": "saas",
  "budget_value": 15000,
  "timeline_months": 1,
  "is_decision_maker": true,
  "form_answers_summary": "Modernisation infra IA generative",
  "email": "cto@vercel.com"
}
```

Exemple de sortie (output)

```
{
  "status": "success",
  "action": "create",
  "score": 92,
  "classification": "sql",
  "ab_variant": "A",
  "firmographics": {"industry": "Software Development", "employees": "100-500"}
}
```

Analyse detaillee (verifiee contre le code reel)

Logging : non conforme. Verifie dans les vrais logs produits (qualifier_logs.json) : {agent_name, action_type, lead_id, score, classification, dimensions_breakdown, model_used, timestamp} -- il manque input_summary, output_summary, client_id et latency_ms (4 des 9 champs obligatoires).

Routes reelles verifiees : POST /form-submission (intake Typeform), POST /event, POST /api/v1/evaluate, POST /api/v1/learning/trigger-analysis, POST /api/v1/learning/trigger-evaluation, POST /api/v1/webhooks/tracking. Ces routes 'learning' semblent concues pour etre declenchees par l'Analyste en Phase C.

Affiche Phase C dans son README (apprentissage par outcomes, A/B testing natif, enrichissement firmographique, evaluation MLOps Langfuse) -- fonctionnalites avancees mais logging de base non conforme, meme schema que Commander/Content Strategist.

Agent 4 : Content & Conversion Strategist

Couche : Acquisition

Definition du role

Le Content Strategist produit tout ce qu'un prospect lit ou voit avant de soumettre un formulaire : copie publicitaire, pages d'atterrissage, briefs creatifs. Il possede le CRO (Conversion Rate Optimization) de bout en bout : si une page convertit mal, c'est lui qui identifie pourquoi et propose la correction.

Responsabilites

- Generer des variantes de copie publicitaire (headline, primary text, CTA).
- Ecrire des briefs creatifs pour les assets visuels.
- Construire et maintenir la copie des landing pages par client.
- Faire tourner des tests A/B sur les assets de copie.
- Analyser les donnees d'abandon de formulaire et proposer des reformulations.
- Reecrire proactivement les assets sous-performants (donnees venant de l'Analyst).

Ce que cet agent ne fait PAS

- Ne gere aucune campagne publicitaire (role du Media Buyer).
- Ne touche jamais au CRM.
- N'envoie aucun message a un prospect.

Exemple d'entree (input)

```
POST /cro-analysis
{
  "conversion_rate": 12.0
}

(le SEUL champ accepte -- pas d'identifiant de page, pas de client_id)
```

Exemple de sortie (output)

```
{
  "issue": "low_conversion",
  "recommendations": [
    "Strengthen the Hero section with a clearer value proposition.",
    "Make the CTA more visible and action-oriented.",
    "..."
  ]
}
```

Analyse detaillee (verifiee contre le code reel)

Logging : non conforme, schema totalement different du format obligatoire. `json_logger_tool.py log {client_id, version, timestamp, asset_type, content} --` c'est un versionnage de contenu, pas un journal d'action.

README.md vide : zero documentation, alors que le brief exige sa redaction 'day one avant toute logique'.

Limitation structurelle majeure sur /cro-analysis : le modele LandingPagePerformance ne contient

qu'un seul champ, `conversion_rate`. Aucun identifiant de page, aucun `client_id`. Conséquence : même en appelant cette route, Content Strategist ne peut structurellement pas savoir de quelle page il s'agit -- il faudrait ajouter un champ `page_id` cote leur code.

Routes réelles vérifiées (port 8000, même défaut que CRM Keeper -- collision possible) : POST `/ad-copy`, POST `/landing-page`, POST `/creative-brief`, POST `/rewrite` (ad copy pilote par le CTR, pas par le taux de conversion landing page), POST `/cro-analysis`, POST `/form-analysis`.

Un champ `page_id` existe bien dans le code (`models/deployment_request.py`, utilisé en interne par `agent.py/ABTestManager`) mais n'est branché sur aucune route HTTP exposée.

Agent 5 : Media Buyer

Couche : Acquisition

Definition du role

Le Media Buyer gere les campagnes publicitaires sur Meta Ads et Google Ads : ciblage, budgets, encheres, structure de campagne. Son metric de succes n'est pas le clic mais le cout par SQL. Il ne cree jamais de creation publicitaire -- il execute ce que le Content Strategist lui fournit.

Responsabilites

- Creer et gerer les structures de campagnes par client.
- Configurer le ciblage (Lookalike, retargeting CRM).
- Allouer et rythmer les budgets, surveiller les KPI quotidiens (CPM, CPC, CTR, CPL).
- Mettre en pause les ad sets sous-performants (regle du CPL > 2x cible sur 3 jours).
- Deployer les creatifs recus du Content Strategist (rotation champion/challenger 80/20).
- Envoyer un resume de performance a l'Analyst chaque semaine.
- [Phase C] Ajustement automatique des encheres selon le ROAS, dayparting, push CAPI.

Ce que cet agent ne fait PAS

- Ne redige jamais de texte publicitaire ni de visuel.
- Ne stocke aucune donnee client (role du CRM Keeper).
- Ne genere pas de strategie complexe (role de l'Analyst).

Exemple d'entree (input)

```
GET /performance/last-7-days
```

(aucun parametre -- un seul compte publicitaire global, pas de client_id)

Exemple de sortie (output)

```
{
  "status": "success",
  "data": {
    "spend": 342.17,
    "impressions": 58210,
    "clicks": 1032,
    "leads": 47
  }
}
```

(bloc agrege pour tout le compte -- jamais par campagne/ad set/creative asset)

Analyse detaillee (verifiee contre le code reel)

Logging : non conforme, mais le plus proche des 6. Le dict au format exact est construit dans la route POST /ads/alert, mais seulement affiche via print() -- jamais ecrit dans un fichier ni une base. Aucune trace ne survit a un redemarrage.

Routes reelles verifiees (port 8004) : POST /ads/alert (reaction autonome, coupe l'ad set si cpl_spike detecte), GET /performance/last-7-days, GET /campaign/{campaign_id}/summary, POST /crm/push-segment (Lookalike Audiences), POST /ads/rotate-creatives, POST /capi/push-conversion,

GET /test/phase-c/adjust-bid, GET /test/phase-c/dayparting.

Limitation structurelle pour l'attribution : /performance/last-7-days ne donne qu'un SEUL bloc agrege pour tout le compte (level='account' dans l'appel Graph API), jamais decompose par campagne, ad set ou creative asset. Le B2 (attribution) de l'Analyst ne peut donc pas etre alimente par cette route.

Mode d'echec ambigu : si l'appel interne a l'API Meta echoue, le endpoint repond quand meme HTTP 200 avec des zeros (spend=0, leads=0) plutot qu'une erreur -- impossible de distinguer 'vraiment zero' de 'l'appel a echoue silencieusement' depuis l'exterieur.

Bon point : retry avec tenacity (stop_after_attempt(3), wait_exponential) deja en place sur les appels externes, seul agent avec Analyst a l'avoir fait avant qu'on le signale.

Agent 6 : Closer

Couche : Conversion

Definition du role

Le Closer prend un lead qualifie et le pousse jusqu'au rendez-vous booke, sur WhatsApp (conversationnel, immediat) et email (sequences structurees). Il suit une bibliotheque de templates pre-approuves -- le LLM remplit les variables et choisit le bon template, il n'improvise pas librement (sauf en Phase C, encadre par une analyse de sentiment).

Responsabilites

- Executer des sequences d'emails automatisees (Nurture MQL, Follow-up SQL) via Brevo.
- Envoyer des messages WhatsApp sortants via 360dialog (templates et messages libres).
- Gerer les reponses entrantes WhatsApp via webhook.
- [Phase C] Analyser le sentiment et generer des contre-arguments en temps reel (LLM).
- Gerer la prise de rendez-vous via les webhooks Cal.com.
- Executer une relance quotidienne des prospects silencieux depuis 7/30/60 jours.

Ce que cet agent ne fait PAS

- Ne stocke aucune donnee de maniere permanente -- tout est delegue au CRM Keeper.
- Ne qualifie pas les leads (suppose que le Qualifier l'a deja fait).
- N'orchestre pas le systeme (c'est le Commander qui declenche la premiere etape).

Exemple d'entree (input)

```
POST /whatsapp/incoming
{
  "entry": [{"changes": [{"value": {"messages": [{"
    "from": "+33612345678",
    "text": {"body": "Interesse mais integration trop complexe pour l'equipe."}
  ]}}]}]}
}
```

Exemple de sortie (output)

```
{
  "status": "success",
  "message": "Traitement en cours"
}
```

(traitement reel delegue en tache de fond : analyse de sentiment, generation de reponse, envoi WhatsApp, mise a jour CRM Keeper)

Analyse detaillee (verifiee contre le code reel)

Logging : non conforme -- le pire des 6. Aucune fonction ne construit jamais le format obligatoire, nulle part dans le code (ni api.py, ni agent.py, ni les tools/). Tout passe par des print() et des traces Langfuse (tags/metadata), sans jamais respecter le contrat.

Bug reel trouve : dans tools/sequence_runner.py::update_lead_via_crm_keeper, un probleme d'indentation fait que le bloc with propagate_attributes(...) ne contient que la ligne url = ... -- l'appel HTTP reel (celui qui compte pour la trace) s'execute EN DEHORS du contexte de tracage. Le CRM est bien

mis à jour, mais le tag Langfuse pour cette action précise n'est jamais rattaché correctement.

Routes réelles vérifiées (port 8002) : POST /api/v1/closer/trigger, POST /whatsapp/incoming, POST /webhook/cal (BOOKING_CREATED et MEETING_ENDED), POST /webhooks/hubspot.

Point positif : les timestamps last_action_at écrits dans le CRM Keeper à chaque message sont exactement la donnée brute dont les métriques time_to_first_contact et response_rate de l'Analyst ont besoin -- bonne conception du flux de données malgré le logging manquant.

Déjà en Phase C fonctionnelle (analyse de sentiment + escalade + Guardrails AI) alors que son propre README affiche encore 'Phase B'.

Agent 7 : Analyst (cet agent -- construit dans ce depot)

Couche : Intelligence

Definition du role

L'Analyst est le seul agent qui regarde en arriere. Les 6 autres agissent dans le present ; l'Analyst etudie ce qui s'est deja passe : il calcule les 9 KPI canoniques, identifie les patterns de conversion, detecte les anomalies avec un plancher anti-bruit, et genere des recommandations concretes chaque semaine -- jamais executees par lui-meme, seulement transmises.

Responsabilites

- Calculer les 9 KPI canoniques (CPL, CPQ, CPQL, CPBD, ROAS, taux de conversion par etape, temps de premier contact, taux de reponse, taux de presence RDV).
- Repartir CPL/CPQ/CPQL par campagne/ad set/creative asset (B2), plancher SQL toujours visible.
- Flaguer les landing pages sous 15% de conversion (B3) et notifier Content Strategist.
- Detecter les anomalies de conversion avec plancher de volume anti-bruit (ANA-03).
- Generer un rapport hebdomadaire d'optimisation avec resume en langage naturel (DeepSeek).
- Servir de binome d'observation pour les autres agents avant qu'ils agissent.
- Tracer chaque action via Langfuse, journaliser chaque action au format obligatoire.

Ce que cet agent ne fait PAS

- Ne change jamais un budget, n'envoie jamais un message.
- Ne modifie jamais une fiche CRM directement.
- Ne publie jamais de contenu.
- Ne bypassse jamais l'approbation du Commander ou les guardrails d'un agent spécialiste.

Exemple d'entree (input)

```
GET /report
```

(ou commande Telegram /report, /weekly_report, /optimisation_report)

Exemple de sortie (output)

```
{
  "cpl": {"value": 46.72, "source": "live", "previous_value": 40.0},
  "roas": {"value": 1.6, "source": "simulated"},
  "stage_conversion_rate": {"value": 34.2, "source": "live"}
  ...
}
```

(source = "live" quand CRM Keeper + Media Buyer sont joignables, sinon repli propre sur les donnees simulees -- jamais de melange partiel)

Analyse detaillee (verifiee contre le code reel)

Logging : conforme. Format exact, JSONL local + table Postgres agent_logs testee contre un vrai serveur Postgres 17, resilience verifiee (base coupee = l'agent continue de tourner).

Donnees reelles : CPL/CPQ/CPQL/stage_conversion_rate branches en direct sur CRM Keeper et

Media Buyer (repli automatique et complet sur donnees simulees si l'un des deux est injoignable). ROAS et l'attribution B2 restent simules en permanence -- verifie qu'aucune des deux APIs amont n'expose la granularite necessaire.

Resilience : chaque appel externe (DeepSeek, Langfuse, CRM Keeper, Media Buyer, Content Strategist) passe par httpx/litellm + tenacity (3 tentatives, backoff exponentiel), et ne leve jamais d'exception au-dela de son propre module -- degradation systematique vers "no data", jamais de crash.

Tests : 81 tests automatises, suite hermetique (aucun appel reseau reel pendant les tests, meme avec de vraies cles configurees localement).

Phase A et Phase B completes selon le work plan (SETUP-03, B1-B7, ANA-03), Phase C non commencee -- decision volontaire, en attente de priorisation.

Synthese

Sur les 7 agents du systeme, seuls l'Analyst et le CRM Keeper respectent integralement le format de log obligatoire du cahier des charges. Les 5 autres presentent chacun un ecart structurel different : logging jamais persiste (Media Buyer), jamais construit (Closer), champs faux (Commander), schema completement different (Content Strategist), ou champs manquants (Qualifier).

Trois limitations structurelles cote API des agents existants bloquent une integration complete de l'Analyst avec des donnees 100% reelles :

- CRM Keeper n'expose aucun champ de valeur de deal -> le ROAS reste simule.
- Media Buyer n'expose la depense qu'en un seul bloc pour tout le compte -> l'attribution par campagne/ad set/creative asset reste simulee.
- Content Strategist n'a aucun identifiant de page dans son API -> la notification de page sous-performante ne peut pas etre correlee de leur cote.

Ces trois points ne sont pas resolubles unilateralement depuis le code de l'Analyst : ils necessitent une evolution des schemas de donnees exposes par CRM Keeper, Media Buyer et Content Strategist.